

Volumes and Networks in Docker Compose

Volumes in Docker Compose

What are Volumes?

By default, data inside a Docker container is lost when the container is removed. Volumes are a mechanism to persist data outside the container's filesystem so that it survives container restarts and removals.

In Docker Compose, volumes can be defined at two levels:

- Service-level — specifying where to mount the volume inside a specific container
- Top-level — declaring named volumes that Docker manages and that can be shared across multiple services

Types of Volume Mounts

1. Named Volumes (Recommended for persistent data)

services:

db:

image: postgres:15

volumes:

- db-data:/var/lib/postgresql/data

volumes:

db-data:

Here, db-data is a named volume managed by Docker. The data stored at /var/lib/postgresql/data inside the container is persisted in this volume. Even if the container is removed and recreated, the data remains.

2. Bind Mounts (Used for development — mounts a host directory)

services:

app:

image: node:18

volumes:

- ./src:/app/src

This mounts the ./src folder from your host machine into /app/src inside the container. Changes made to files on the host are immediately reflected inside the container, making this very useful during development.

3. Anonymous Volumes

services:

app:

volumes:

- /app/node_modules

Anonymous volumes are created without a name. They are less portable and harder to manage, so named volumes are preferred.

Volume Options

volumes:

- db-data:/var/lib/postgresql/data:ro

The :ro suffix makes the volume read-only inside the container. :rw is the default (read-write).

Networks in Docker Compose

What are Compose Networks?

When Docker Compose starts your services, it automatically creates a default network and connects all services to it. This means all services in the same Compose file can reach each other by their service name as the hostname.

However, you can define custom networks for more control — for example, to isolate certain services from each other.

Default Network Behaviour

When you run docker compose up, Docker Compose creates a network named <project-name>_default and attaches all services to it. No additional configuration is needed for basic inter-service communication.

Defining Custom Networks

services:

frontend:

image: nginx

networks:

- frontend-net

backend:

image: node:18

networks:

- frontend-net
- backend-net

db:

image: postgres:15

networks:

- backend-net

networks:

frontend-net:

backend-net:

In this example:

- The frontend service can talk to backend because they share frontend-net
- The frontend service cannot directly reach db because they are on different networks
- The backend service acts as a bridge — it is on both networks and can talk to both frontend and db
- This is a security best practice — the database is not exposed to the frontend layer

Network Driver Types

- bridge — the default driver for single-host networking; containers on the same bridge network can communicate
- host — the container shares the host's network stack directly; no network isolation
- overlay — used in Docker Swarm for multi-host networking

- none — disables all networking for a container

networks:

backend-net:

driver: bridge

Connecting to External Networks

If you have a network created outside of Compose (e.g., by another Compose project), you can connect to it:

networks:

shared-net:

external: true

Note 3: Environment Variables, and the build vs image Fields

Environment Variables in Docker Compose

Environment variables are used to pass configuration values into containers at runtime without hardcoding them into the application code or Docker image. This is essential for values like database passwords, API keys, hostnames, and ports.

Method 1: Inline in docker-compose.yml

services:

app:

image: myapp:latest

environment:

NODE_ENV: production

DB_HOST: db

DB_PORT: 5432

DB_PASSWORD: mysecretpassword

This works but is not ideal for sensitive values like passwords since they are visible in the YAML file.

Method 2: Using an .env File

Docker Compose automatically reads a file named `.env` in the same directory as your `docker-compose.yml` and substitutes the values using `${VARIABLE_NAME}` syntax.

`.env` file:

```
DB_PASSWORD=mysecretpassword
```

```
DB_USER=admin
```

```
APP_PORT=3000
```

`docker-compose.yml`:

services:

app:

```
image: myapp:latest
```

ports:

```
- "${APP_PORT}:3000"
```

environment:

```
DB_PASSWORD: ${DB_PASSWORD}
```

```
DB_USER: ${DB_USER}
```

Important: The `.env` file should be added to `.gitignore` so that sensitive credentials are not committed to source control.

Method 3: `env_file` Directive

You can specify a custom environment file per service:

services:

app:

```
image: myapp:latest
```

`env_file`:

```
- ./config/app.env
```

```
- ./config/db.env
```

Each file contains `KEY=VALUE` pairs, one per line.

Passing Host Environment Variables

You can pass a variable from the host shell into the container by listing the key without a value:

environment:

- MY_SECRET_KEY

Docker Compose will look for MY_SECRET_KEY in the host shell environment and pass it into the container.

The build vs image Fields

When defining a service in Docker Compose, you must tell it which container image to use. There are two ways to do this.

Using image

The image field tells Compose to pull a pre-built image from Docker Hub or another registry.

services:

db:

image: postgres:15

This is used when:

- You are using an official or third-party image as-is
- The image is already built and published to a registry
- You don't need to customise the image build process

Using build

The build field tells Compose to build the image locally from a Dockerfile before running the container.

services:

app:

build: ./app

This is shorthand for:

services:

app:

build:

context: ./app

dockerfile: Dockerfile

- context is the directory that Docker uses as the build context (files available during the build)
- dockerfile specifies the name of the Dockerfile if it is not the default Dockerfile

You can also pass build arguments:

services:

app:

build:

context: ./app

args:

NODE_VERSION: 18

APP_ENV: production

Using Both build and image Together

You can specify both — build tells Compose how to build the image, and image gives the resulting image a name and tag:

services:

app:

build: ./app

image: myorg/myapp:latest

This is useful when you want to build the image locally and also push it to a registry with a specific name.

When to Use Which

Situation	Use
Using official database image	image
Custom application with a Dockerfile	build
Pre-built app image in a registry	image
Development with live rebuilds	build
Production deployment from registry	image